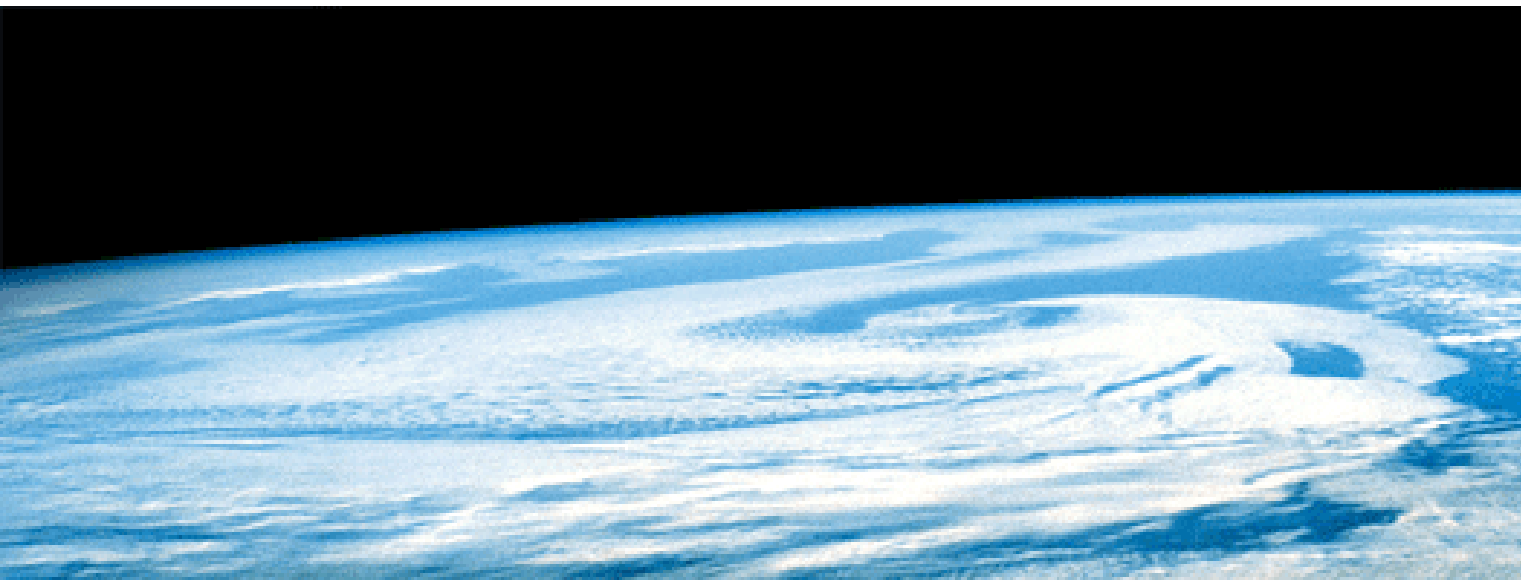




Advanced Servlets and Java Server Pages[tm] Programming

Doris Chen Ph.D.
Staff Engineer
Technology Evangelism
Sun Microsystems, Inc.





Chapter 7

Best Practices

in JSP[tm]/Servlets Programming

Objectives

- Learn best practices for JSP[tm] and servlets programming

Avoid String Concatenation "+=" (all)

- **String** objects are **immutable**
- String **concatenation** creates multiple, intermediate representations
- Use **mutable StringBuffer** for heavy-duty string manipulation

```
String badStr = new String();
StringBuffer goodBuff = new StringBuffer(1000);
for (int i=0; i<1000; i++) {
    badStr += myArray[i]; //creates new Strings
    goodBuff.append(myArray[i]); //same buffer
    String goodStr = new String(goodBuff);
}
```

Avoid Overhead of Acquiring and Closing JDBC Connection (All)

- Avoid acquiring connections directly from JDBC driver
- Use **JDBC connection pooling** in JDBC2.0
 - `DataSource.getConnection()`

```
public void doGet(...)  
{...  
    Connection conn=DriverManager.getConnection(...);  
    PreparedStatement pStmt = conn.prepareStatement("...");  
    ResultSet rs = pStmt.executeQuery();    ...
```

Wrong Way!!!

```
private DataSource ds = null;  
public void doGet(...)  
{...  
    Connection conn=ds.getConnection("db2admin",  
        "db2admin");  
    PreparedStatement pStmt = conn.prepareStatement("...");  
    ResultSet rs = pStmt.executeQuery();    ...
```

Right Way!!!

Avoid Overhead of Acquiring and Closing JDBC Connection (All) (cont.)

- Failing to close JDBC connections can cause other users to wait for connections
- **Close** JDBC statements when you are through with them
- Ensure your code to close **all JDBC resources** even in **exception** and **error conditions**

Right Way!!!

```
Connection conn =null;
ResultSet rs =null;
PreparedStatement pss=null;
//Create all JDBC resources here and then release them
    when done.
Try {if (pss != null) pss.close(); }
catch (Exception e) {}

try {if (conn!= null) conn.close(); }
catch (Exception e) {}
```

Reuse DataSource for JDBC connections (all)

- Make JDBC connections by obtaining `javax.sql.DataSource` in JDBC 2.0
 - `DataSource.getConnection()`
- **Avoid overhead** of acquiring a `javax.sql.DataSource` for each SQL access

```
public void doGet(...)
```

Wrong Way!!!

```
{ ...  
    ctx = new InitialContext(...)  
    DataSource ds = (DataSource)ctx.lookup("jdbc/SAMPLE");  
    ctx.close();  
    Connection conn = ds.getConnection("db2admin",  
        "db2admin");  
    PreparedStatement pStmt = conn.prepareStatement("...");  
    ResultSet rs = pStmt.executeQuery();  
    ...
```

Reuse DataSource for JDBC connections (all) (cont.)

- Acquire `javax.sql.DataSource` in `init()` method and maintain it in a common location for **reuse**

```
//Caching DataSource which is obtained from init()
```

```
private DataSource ds = null;
```

```
public void init(...)
```

```
{...
```

```
    Context ctx = null;
```

```
    try {
```

```
        ...
```

```
        ctx = new InitialContext(...)
```

```
        ds = (DataSource)ctx.lookup("jdbc/SAMPLE");
```

```
        ctx.close();
```

```
    } catch ...
```

```
}
```

Right Way!!!

Minimize Use of System.out.println (all)

- `System.out.println` **synchronize** processing for the duration of disk I/O
 - Can significantly **slow** throughput
- Avoid using indiscriminate `System.out.println`
- Use a "**final boolean**" to control error and debugging situations

Right Way!!!

```
public SomeServlet extends HttpServlet {  
    private final static boolean DEBUG = false;  
    public void doGet(...) {  
        if (DEBUG) {  
            System.out.println("An error has...");  
        }  
    }  
}
```

Release HttpSession When Finished (servlets)

- Only a certain number of HttpSession can be maintained in memory
- Use HttpSession.invalidate() to release session resource

Right Way!!!

```
public class ReceiptServlet extends HttpServlet {  
    public void doPost ...{  
        ...  
        ShoppingCart scart = (ShoppingCart)  
            session.getAttribute("examples.bookstore.cart");  
        ...  
        // Clear out shopping cart by invalidating the session  
        session.invalidate();  
    }  
}
```

Use `HttpServlet.init()` to Perform Expensive Operations (servlets)

- `Servlet.init()` only invoked **once** when servlet is first loaded
- Use `HttpServlet.init()` to perform expensive operations that need only be done once
 - `init()` is thread-safe
 - `init()` can be cached safely in servlet instance variables

Don't Create HttpSession in JSPs by Default (JSP)

- By **default** JSP creates HttpSession
 - HttpSession is an implicit object
- If do not need HttpSession in JSP, turn it to **false**
 - Save some performance overhead
- `<%@page session="false"%>`

Avoid Beans.instantiate() to Create New Bean (servlets)

- Beans.instantiate() create a new bean
 - By retrieving a serialized bean
 - Or creating a new bean
 - File system checks for a serialized bean
- Avoid overhead by simply using **new** to create bean object

```
public void doGet(...)  
{...  
    PingBean ab = (PingBean)Beans.instantiate(...);  
    ...}
```

Wrong Way!!!

```
public void doGet(...)  
{...  
    PingBean ab = new PingBean();  
    ...}
```

Right Way!!!

Summary

- Various best practices can be used in JSP/servlets programming to improve application performance



Doris Chen Ph.D.
doris.chen@sun.com

